

**PROJECT REPORT OF INDUSTRY ORIENTED HANDS ON EXPERIENCE
(IOHE)**

ON

CIE-VAULT

(Copyright Information Extraction Vault)

Submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted By

Tanish – 2211981428

Tanish Kumar – 2211981429

Sahil Akhtar – 2211981330

Sourabh Kumar – 2211981401

Under the Guidance of

Dr. Swati Goel

Professor

Department of Computer Science and Engineering

CHITKARA UNIVERSITY

Punjab, India

ACKNOWLEDGEMENT

We express our sincere gratitude to our respected mentor Dr. Swati Goel for her continuous guidance, encouragement, and valuable suggestions throughout the development of the project "CIE-Vault". Her technical expertise and mentorship played a significant role in shaping the direction of this project.

We are also thankful to the Department of Computer Science and Engineering, Chitkara University, for providing the infrastructure, resources, and learning environment necessary for successful project development.

We would also like to acknowledge our teammates for their cooperation, technical discussions, testing efforts, and dedication during every phase of development. Team collaboration enabled us to overcome technical challenges and build a secure and scalable research management platform.

Finally, we express our gratitude to our friends and family members for their motivation and support throughout the project lifecycle.

ABSTRACT

The increasing number of research-oriented projects and innovation-based academic activities has significantly increased the need for secure intellectual property management systems within educational institutions. Traditional methods of copyright filing and document sharing are often dependent on manual workflows, insecure communication channels, and unorganized storage systems. To address these challenges, CIE-Vault (Copyright Information Extraction Vault) is proposed as a secure MERN-stack based academic copyright management platform designed specifically for educational institutions. The platform automates the complete workflow of copyright submission, mentor review, approval tracking, plagiarism analysis, and secure document sharing. The system integrates Automated Metadata Extraction using Node.js and pdf-parse technology to extract important information such as filing numbers, project titles, and copyright details directly from uploaded government receipts. OCR-based processing is additionally used for scanned documents, reducing manual data entry and minimizing operational errors. CIE-Vault introduces a Hybrid Access Control mechanism where internal mentors receive controlled access to confidential data while external reviewers can access only public research abstracts through a restricted Global Research module. AES-256 encryption, JWT authentication, HTTP-only cookies, bcrypt hashing, and secure document preview systems collectively ensure the protection of sensitive intellectual property. The platform also integrates a Hybrid Plagiarism Detection Engine using local similarity algorithms and AI-assisted evaluation methods to strengthen academic integrity. Developed using React.js, Tailwind CSS, Node.js, Express.js, and MongoDB, the system provides a responsive and scalable architecture suitable for future expansion.

TABLE OF CONTENTS

1.	Acknowledgement	2
2.	Abstract	3
3.	Introduction	5
4.	Problem Background	6
5.	Objectives of the Project	7
6.	Proposed System	8
7.	Key Features	9
8.	Technology Stack	10
9.	Methodology	11
10.	Security Implementation	12
11.	Project Folder Structure	13
12.	Workflow Diagrams	14
13.	Use Case Diagram	16
14.	Class Diagram	17
15.	System Snapshots & Code	18
16.	Copyright Receipt Details	25
17.	Expected Outcomes and Benefits	26
18.	Conclusion and Future Scope	27
19.	References	28

1. INTRODUCTION

The rapid growth of academic research, software innovation, and student-led projects has increased the importance of intellectual property management in educational institutions. Students frequently create unique projects, research papers, software systems, and innovative solutions that require copyright registration and secure handling. However, the current workflow followed in many institutions remains highly fragmented, manual, and insecure.

Traditionally, students submit copyright receipts and project reports through emails, messaging platforms, or cloud storage links. These approaches create several operational and security-related issues such as unauthorized access, accidental file deletion, poor organization, lack of transparency in mentor reviews, and improper tracking of submissions.

In many cases, students are also required to share confidential government portal credentials with mentors to manage copyright filings. This introduces serious privacy concerns and increases the possibility of misuse or unauthorized access.

CIE-Vault is developed as a centralized and secure academic copyright management platform designed to eliminate these limitations. The system automates submission workflows, protects sensitive credentials, ensures secure collaboration, and provides institutional transparency.

The platform follows a modern full-stack architecture using React.js, Node.js, Express.js, MongoDB, AES-256 encryption, JWT authentication, OCR processing, and AI-assisted plagiarism detection.

2. PROBLEM BACKGROUND

Current copyright filing systems in academic institutions suffer from several major limitations:

- Insecure sharing of government portal credentials through unprotected channels.
- Absence of centralized document repositories for research records.
- Manual metadata entry resulting in human errors and inconsistencies.
- Weak access control mechanisms and poor authorization management.
- Difficulty in plagiarism verification and originality checks.
- Lack of workflow transparency between students and mentors.
- No secure collaboration model for academic research.
- Improper approval tracking and submission status management.
- Difficulty in managing institutional research at scale.

These limitations result in inefficient workflows, increased human error, poor document organization, and significant security vulnerabilities that compromise intellectual property protection.

3. OBJECTIVES OF THE PROJECT

The primary objective of CIE-Vault is to build a secure, intelligent, and scalable academic copyright management ecosystem. The following objectives guide the system design and implementation:

1. Automate metadata extraction from uploaded copyright receipts using pdf-parse and OCR.
2. Eliminate manual filing data entry and reduce human errors in the workflow.
3. Protect sensitive credentials and research documents through AES-256 encryption.
4. Enable secure and transparent collaboration between students and mentors.
5. Provide hybrid plagiarism verification mechanisms using AI and local algorithms.
6. Maintain transparency in approval workflows with real-time notification systems.
7. Improve institutional research management through centralized platforms.
8. Support secure external research visibility with role-based access control.
9. Develop a modular and scalable architecture for future expansion.
10. Create a centralized repository for copyright records and academic submissions.

4. PROPOSED SYSTEM

CIE-Vault is designed as a MERN-stack web platform that digitizes the complete copyright filing lifecycle. The system is structured into three distinct layers, each responsible for specific aspects of the workflow.

4.1 Frontend Layer

The frontend is developed using React.js with TypeScript for type safety, Tailwind CSS for responsive styling, Vite as the build tool, and SWR for data fetching. The frontend provides responsive dashboards for students and mentors, authentication interfaces, file upload systems, real-time notification displays, workflow tracking panels, and secure document preview capabilities.

4.2 Backend Layer

The backend is built using Node.js and Express.js following a middleware-based REST API architecture. The backend handles authentication, file processing, metadata extraction, encryption, workflow management, notification dispatch, and plagiarism detection orchestration.

4.3 Database Layer

MongoDB is used as the primary NoSQL database with Mongoose ODM for schema enforcement. The database stores user records, copyright submissions, metadata, notification logs, access requests, and plagiarism reports in a structured and query-able format.

5. KEY FEATURES

5.1 Automated Metadata Extraction

The system automatically extracts filing numbers and project titles from uploaded receipts using pdf-parse and Tesseract OCR techniques. For scanned documents, OCR processes the image to identify key information, significantly reducing manual data entry errors.

5.2 Hybrid Access Control

Internal mentors receive full access to confidential research data and encrypted reports. External reviewers can only access public research abstracts through the restricted Global Research module. This hybrid model ensures both transparency and security.

5.3 Encrypted File Vault

Uploaded reports and copyright credentials are encrypted using AES-256 before being stored in the private filesystem directory. Files are never stored in publicly accessible locations, and decryption only occurs during authorized secure previews.

5.4 Plagiarism Detection Engine

The platform integrates a hybrid plagiarism detection system combining local Jaccard similarity comparison (using Shingling tokenization) and AI-assisted analysis through the Gemini AI API. Mentors and admins can trigger plagiarism checks directly from the dashboard.

5.5 Notification Engine

Students and mentors receive real-time notifications regarding submission approvals, rejections, plagiarism results, and access requests. The notification system ensures constant communication throughout the workflow lifecycle.

5.6 Secure Preview System

Mentors can preview encrypted research reports securely without downloading confidential files. The system decrypts the document in memory and streams the PDF directly into the browser using authorized endpoints, preventing unauthorized file access.

6. TECHNOLOGY STACK

CIE-Vault employs a comprehensive set of modern technologies across all layers of the application:

Layer	Technologies
Frontend	React.js, TypeScript, Tailwind CSS, Vite, SWR
Backend	Node.js, Express.js, REST APIs, Middleware Architecture
Database	MongoDB (NoSQL) with Mongoose ODM
Security	AES-256 Encryption, JWT Auth, bcrypt, HTTP-only Cookies, Helmet.js
Utilities	pdf-parse, Tesseract OCR, Multer, Gemini AI API
DevOps	Git, npm, Vite bundler, ESLint, TypeScript compiler

7. METHODOLOGY

The project follows an Agile and Iterative software development model, allowing incremental feature delivery and continuous integration of feedback throughout the development lifecycle.

7.1 Development Approach

- Frontend development was performed using React.js with reusable, component-driven UI architecture and Tailwind CSS styling system.
- Backend APIs were implemented using Node.js and Express middleware architecture following RESTful design principles.
- MongoDB was used for metadata storage, workflow state tracking, and notification management.
- Uploaded files are validated for size and format, encrypted using AES-256, and processed through OCR and metadata extraction services before storage.

7.2 Core Processing Pipeline

- Student uploads receipt and project report through the submission interface.
- Frontend validates file size ($\leq 2\text{MB}$ for reports, $\leq 500\text{KB}$ for receipts) and format.
- Multer handles files in memory buffer on the backend server.
- pdfExtractor scans the receipt and OCR extracts key metadata such as filing numbers.
- FileEncryptionService encrypts files using AES-256 before writing to private storage.
- Copyright record is created in MongoDB with extracted metadata and encrypted paths.
- NotificationService dispatches alerts to the assigned mentor.
- Mentor reviews, approves or rejects the submission from the secure dashboard.

8. SECURITY IMPLEMENTATION

Security is the foundational principle of CIE-Vault. Every component of the system is designed with security-first practices to protect intellectual property and user credentials.

8.1 Encryption

- AES-256 encryption protects all sensitive research documents and copyright credentials stored in the system.
- Files are encrypted before being written to disk and decrypted only in memory during authorized access.

8.2 Authentication & Authorization

- JWT (JSON Web Token) authentication provides stateless and secure session handling.
- HTTP-only cookies prevent JavaScript-based token theft and XSS attacks.
- bcrypt hashing with salt rounds protects user passwords against brute-force attacks.
- Role-based access control (RBAC) enforces permissions for Student, Mentor, and Admin roles.

8.3 Network & Application Security

- Helmet.js secures HTTP response headers against common web vulnerabilities.
- Rate limiting prevents excessive API requests and mitigates login abuse attacks.
- File validation checks prevent uploading of malicious or unsupported file types.
- Private file storage ensures uploaded documents are never publicly accessible.

9. WORKFLOW DIAGRAMS

9.1 Plagiarism Detection Sequence Diagram

The following sequence diagram depicts the complete plagiarism detection process. The Mentor navigates to the plagiarism checking page, selects documents, and initiates the check. The backend uses the PlagiarismService to either call the Gemini AI API for AI-based analysis or perform a local Jaccard Overlap computation. Results are returned and displayed on the dashboard.

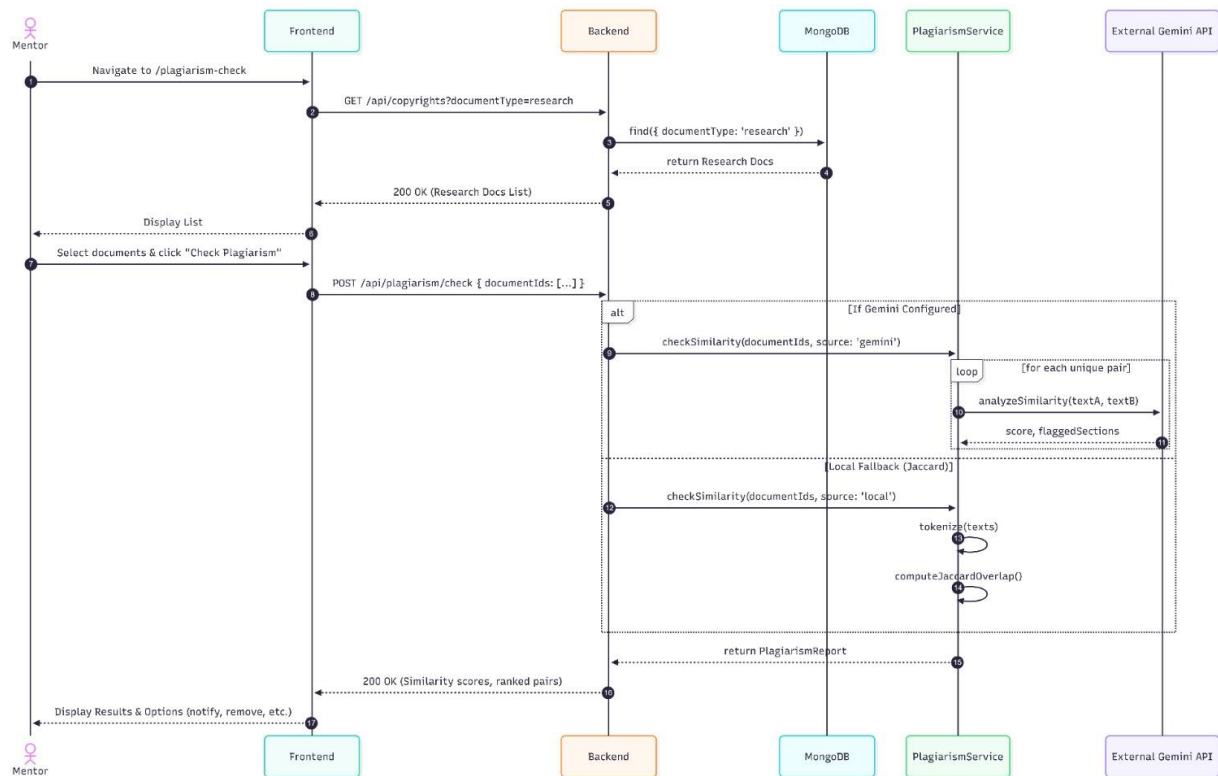


Figure 1 – Plagiarism Detection Sequence Diagram

Initialization (Steps 1–7)

- The Mentor navigates to the plagiarism checking page in the frontend.
- The frontend requests the Backend to retrieve a list of all research papers.
- Backend retrieves files from MongoDB and returns results to the mentor's screen.
- The mentor selects desired files and initiates the plagiarism detection process.

Plagiarism Check Logic (Steps 8–15)

- The backend passes the request to the PlagiarismService with an Alt block decision.
- If Gemini AI is configured, the text is sent to the external Gemini API for similarity analysis.
- Otherwise, a local Jaccard Overlap algorithm tokenizes texts and evaluates percentage overlap.

- Results including similarity scores and flagged sections are returned to the backend.

Results (Steps 16–17)

- PlagiarismService returns similarity scores and ranked pairs to the backend.
- The Frontend displays the results with options to notify, remove, or take further action.

9.2 Student Upload Sequence Diagram

This sequence diagram illustrates the complete student copyright submission pipeline — from client-side file selection and validation through server-side extraction, encryption, database storage, and mentor notification.

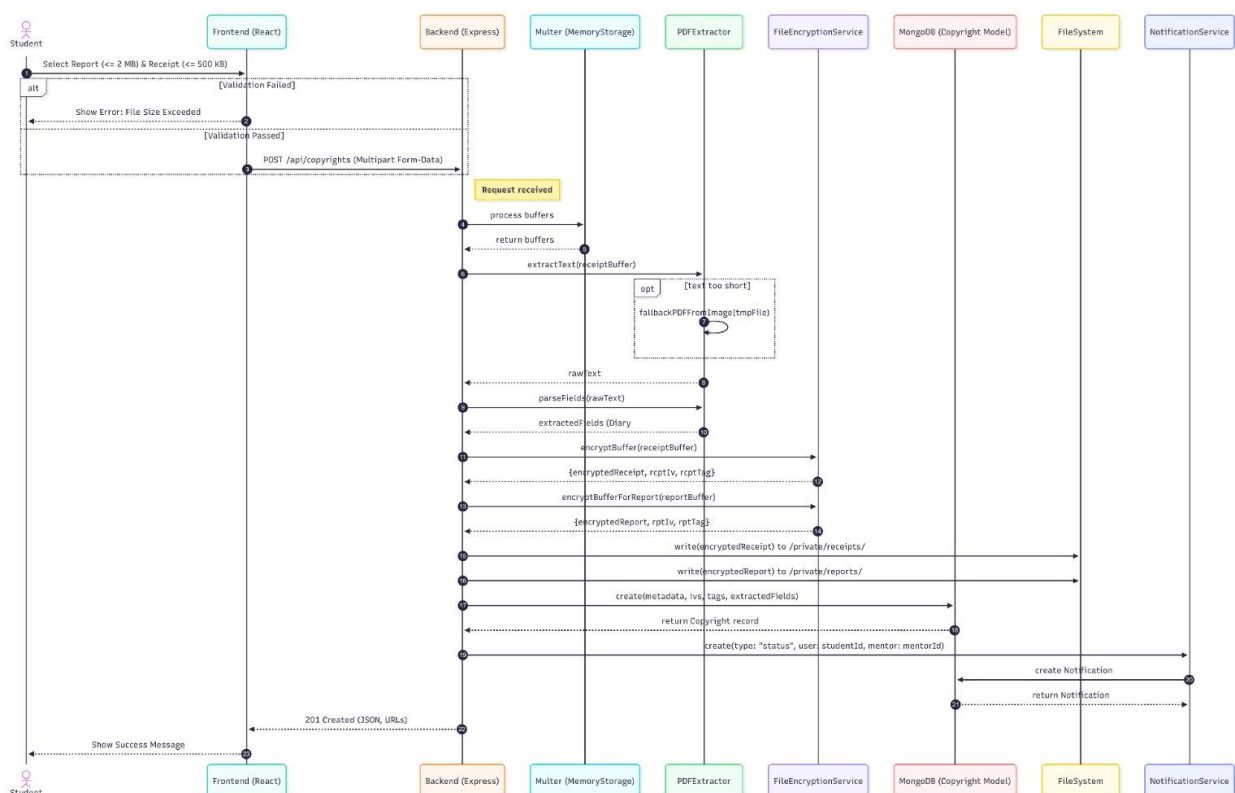


Figure 2 – Student Upload & Submission Sequence Diagram

Upload and Validations

- The student uploads a report and receipt through the dashboard.
- The frontend validates file sizes: report $\leq 2\text{MB}$ and receipt $\leq 500\text{KB}$.
- After validation, files are sent to the backend server through a multipart form request.

Extraction and Encryption

- Multer handles files in memory buffer on the server.
- The pdfExtractor scans the receipt and OCR extracts filing numbers and metadata.
- The FileEncryptionService encrypts both files using AES-256 before storing them.
- Encrypted files are written to the private filesystem directory.

- Copyright record is created in MongoDB with metadata, file paths, and student reference.
- NotificationService creates an alert for the assigned mentor.

9.3 Mentor Review and Approval Workflow

The Mentor Review and Approval Workflow highlights the security measures the application applies while handling file viewing and status changes.

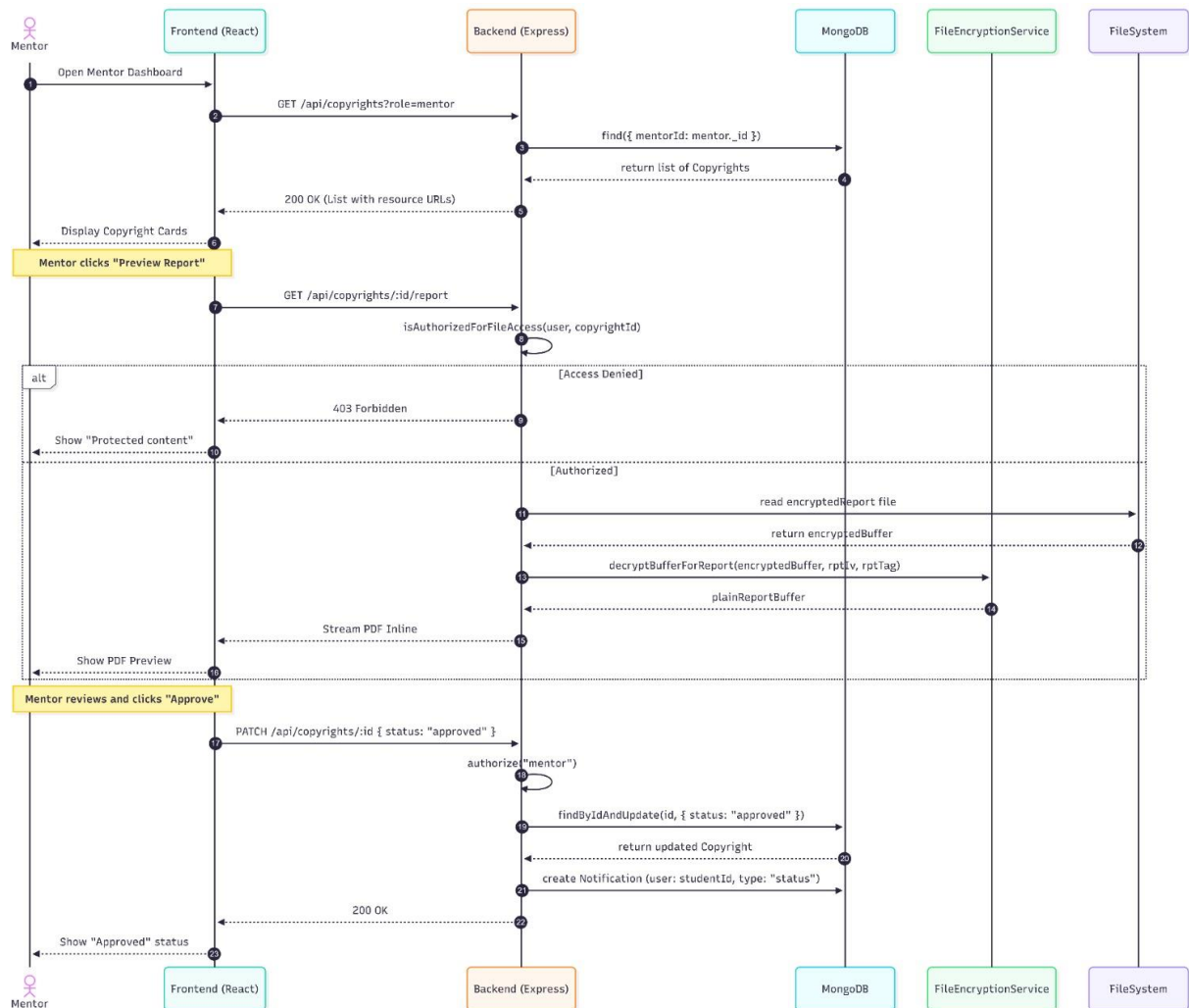


Figure 3 – Mentor Review & Approval Sequence Diagram

Fetching Assigned Work

- Mentor logs into account. The Frontend fetches all submissions allocated to the Mentor ID.
- The Backend retrieves submissions from MongoDB and displays them as "Copyright Cards."

Secure Document Preview

- When the Mentor clicks "Preview Report," the system checks authorization.
- Unauthorized access returns a 403 Forbidden error response.
- Authorized access fetches the encrypted document, decrypts it using FileEncryptionService, and streams the PDF directly into the browser for viewing.

Approval and Notification

- Once the Mentor reviews and clicks "Approve," the frontend sends a PATCH request to the Backend.
- The Backend validates the mentor role and approves the report in MongoDB.
- A notification is automatically created for the student with the updated status.

P.T.O

9.4 System Upload Flowchart

The following flowchart illustrates the complete lifecycle of a copyright submission — from student upload through server-side processing to final mentor approval or rejection.

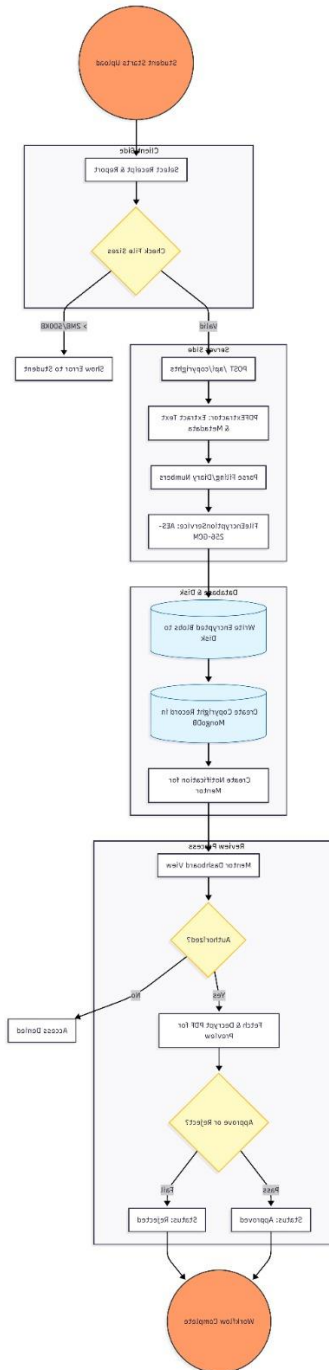


Figure 4 – Complete Upload and Approval Lifecycle Flowchart

6. CLASS DIAGRAM

The class diagram demonstrates the relationships among the core entities of CIE-Vault including User, Copyright, Comment, Notification, FileEncryptionService, and PDFExtractor models.

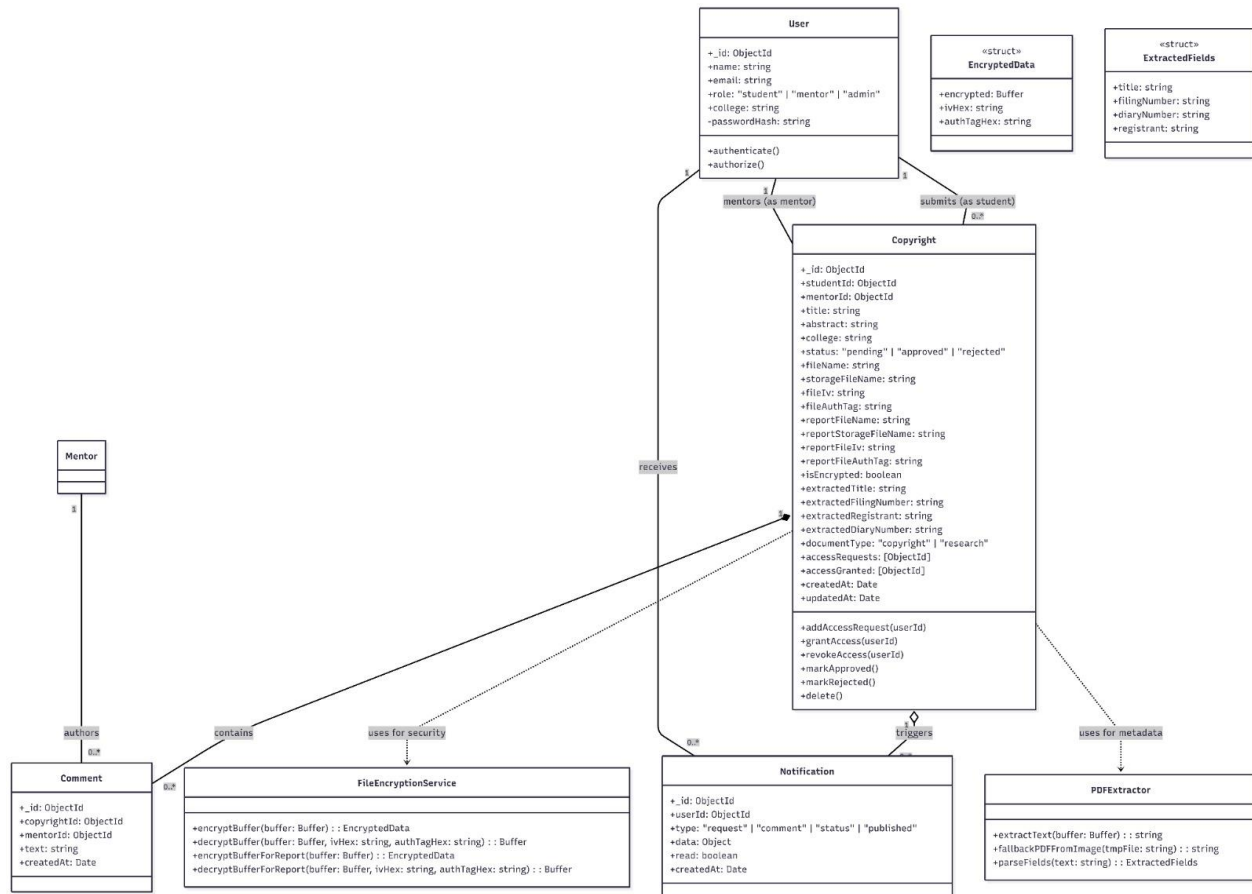


Figure 5 – Class Diagram: Entity Relationships and Services

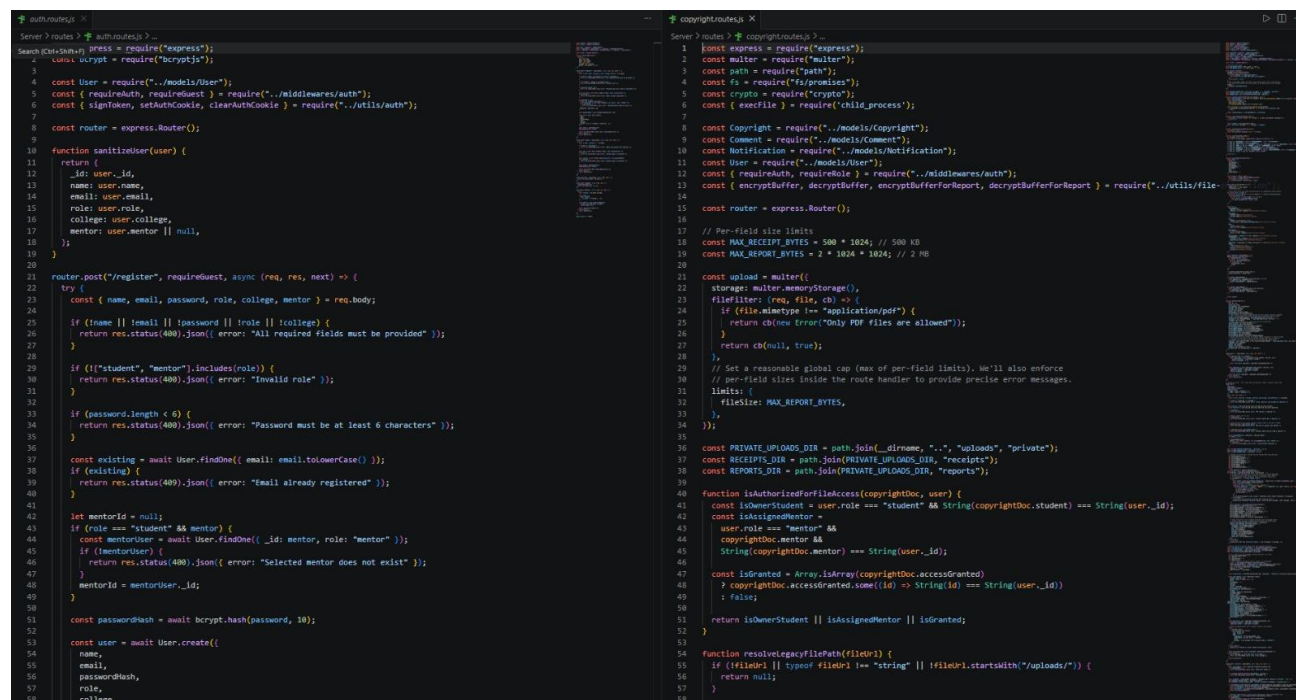
Core Entities

- **User:** Represents both Students and Mentors with role-based attributes and authentication methods.
- **Copyright:** The central submission model containing metadata, encrypted file references, status, and access control lists.
- **Comment:** Linked to a Copyright and Mentor, enabling mentor feedback on submissions.
- **Notification:** Tracks status changes, comments, and access request alerts for users.
- **FileEncryptionService:** Provides AES-256 encryption and decryption for all sensitive file buffers.
- **PDFExtractor:** Extracts raw text, parses filing numbers, and runs OCR fallback for scanned receipts.

11. SYSTEM SNAPSHOTS & CODE

11.1 Landing Page

The landing page serves as the main entry point of CIE-Vault, providing users with an overview of the platform features including end-to-end encryption, role-based access, and auto PDF parsing. Users can sign in or get started from this screen.



The image shows a side-by-side comparison of the CIE-Vault application. The left pane displays the source code for the authentication routes, and the right pane displays the source code for the copyright routes. Both panes are set to a dark theme.

```

Server > routes > authRoutes.js > ...
Search (Ctrl+Shift) press = require("express");
const bcrypt = require("bcryptjs");
const User = require("../models/User");
const { requireAuth, requireRequest } = require("../middlewares/auth");
const { signInToken, refreshToken, clearAuthCookie } = require("../utils/auth");
const router = express.Router();

function sanitizeUser(user) {
  return {
    _id: user._id,
    name: user.name,
    email: user.email,
    role: user.role,
    college: user.college,
    mentor: user.mentor || null,
  };
}

router.post("/register", requireRequest, async (req, res, next) => {
  try {
    const { name, email, password, role, college, mentor } = req.body;
    if (!name || !email || !password || !role || !college) {
      return res.status(400).json({ error: "All required fields must be provided" });
    }
    if (!["student", "mentor"].includes(role)) {
      return res.status(400).json({ error: "Invalid role" });
    }
    if (password.length < 6) {
      return res.status(400).json({ error: "Password must be at least 6 characters" });
    }
    const existing = await User.findOne({ email: email.toLowerCase() });
    if (existing) {
      return res.status(409).json({ error: "Email already registered" });
    }
    let mentorId = null;
    if (role === "student" && mentor) {
      const mentorUser = await User.findOne({ _id: mentor, role: "mentor" });
      if (!mentorUser) {
        return res.status(400).json({ error: "Selected mentor does not exist" });
      }
      mentorId = mentorUser._id;
    }
    const passwordHash = await bcrypt.hash(password, 10);
    const user = await User.create({
      name,
      email,
      passwordHash,
      role,
      college,
    });
  } catch (error) {
    next(error);
  }
});

Server > routes > copyrightRoutes.js > ...
1 const express = require("express");
2 const multer = require("multer");
3 const path = require("path");
4 const fs = require("fs/promises");
5 const crypto = require("crypto");
6 const { writeFile } = require("child_process");
7
8 const Copyright = require("../models/Copyright");
9 const Comment = require("../models/Comment");
10 const Notification = require("../models/Notification");
11 const User = require("../models/User");
12 const { requireAuth, requireRole } = require("../middlewares/auth");
13 const { encryptBuffer, decryptBuffer, encryptBufferForReport, decryptBufferForReport } = require("../utils/file");
14
15 const router = express.Router();
16
17 // Per-field size limits
18 const MAX_RECEIPT_BYTES = 500 * 1024; // 500 KB
19 const MAX_REPORT_BYTES = 2 * 1024 * 1024; // 2 MB
20
21 const upload = multer({
22   storage: multer.memoryStorage(),
23   fileFilter: (req, file, cb) => {
24     if (file.mimetype !== "application/pdf") {
25       return cb(new Error("Only PDF files are allowed"));
26     }
27     return cb(null, true);
28   },
29   // Set a reasonable global cap (max of per-field limits). We'll also enforce
30   // per-field sizes inside the route handler to provide precise error messages.
31   limits: {
32     fileSize: MAX_REPORT_BYTES,
33   },
34 });
35
36 const PRIVATE_UPLOADS_DIR = path.join(__dirname, "..", "uploads", "private");
37 const RECEIPT_DIR = path.join(PRIVATE_UPLOADS_DIR, "receipts");
38 const REPORTS_DIR = path.join(PRIVATE_UPLOADS_DIR, "reports");
39
40 function isAuthorIndefiniteAccess(copyrightDoc, user) {
41   const isOwnerStudent = user.role === "student" && String(copyrightDoc.student) === String(user._id);
42   const isAssignedMentor =
43     user.role === "mentor" &&
44     copyrightDoc.mentor &&
45     String(copyrightDoc.mentor) === String(user._id);
46   const isGranted = Array.isArray(copyrightDoc.accessGranted)
47     ? copyrightDoc.accessGranted.some((id) => String(id) === String(user._id))
48     : false;
49   return isOwnerStudent || isAssignedMentor || isGranted;
50 }
51
52 function resolveAccessFilePath(fileId) {
53   if (!fileId || typeof fileId !== "string" || !fileId.startsWith("/uploads/")) {
54     return null;
55   }
56 }

```

Figure 6 – CIE-Vault Landing Page with User Dropdown

P.T.O

11.2 Login & Registration

The login page securely authenticates users using JWT authentication and encrypted credentials. After login, users are redirected to their role-specific dashboard. The registration page allows students and mentors to create accounts with role selection and college assignment.

Figure 7 – Login Page

Figure 8 – Registration Page

11.3 Mentor Dashboard

The mentor dashboard provides a real-time overview of assigned students, pending submissions, approved filings, and global repository access. Mentors can review copyright cards and manage submissions from this central view.

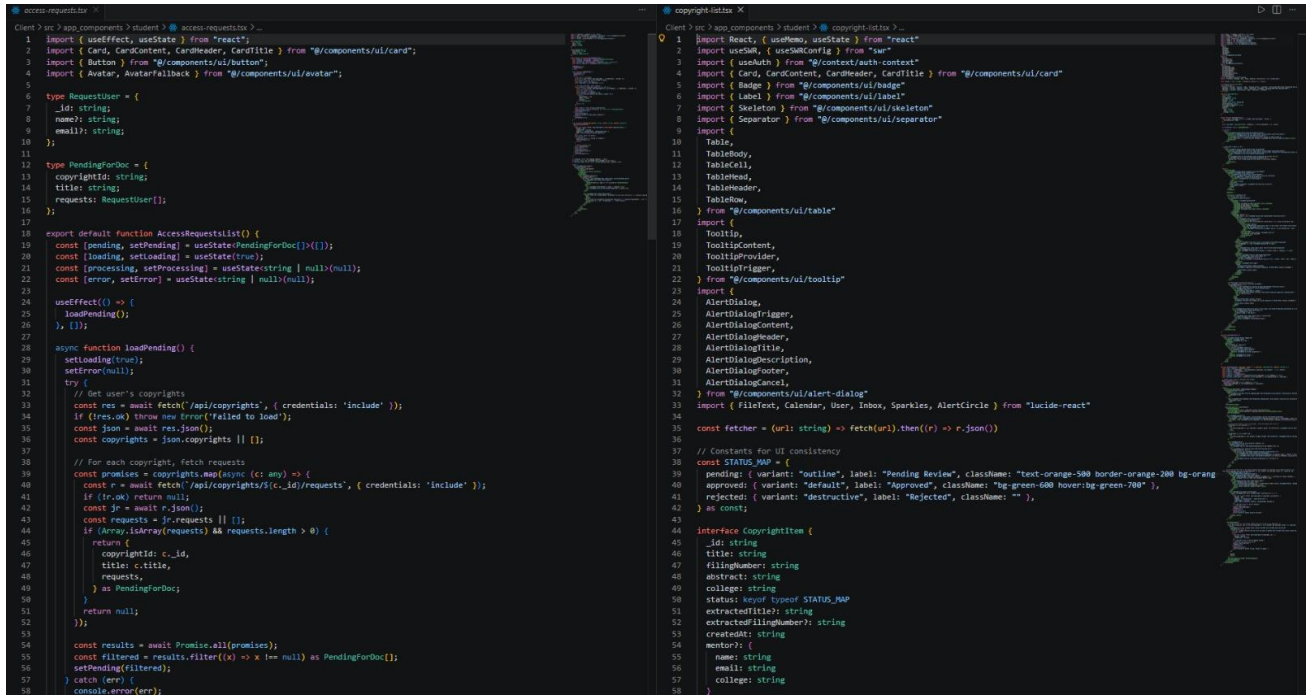


Figure 9 – Mentor Dashboard

11.4 Student Dashboard

The student dashboard enables uploading new copyright submissions (receipt + report), tracking approval status across all filings, and viewing notifications. The portfolio section shows all past filings with statuses: Pending, Approved, or Rejected.

```

1 import { Link } from "react-router-dom";
2 import { Button } from "@components/ui/button";
3 import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
4 import { Badge } from "@components/ui/badge";
5 import { Separator } from "@components/ui/separator";
6 import { Navbar } from "../navbar";
7 import { useAuth } from "@context/auth-context";
8 import {
9   Shield,
10   Upload,
11   Eye,
12   Lock,
13   FileText,
14   Users,
15   Globe,
16   ArrowRight,
17   CheckCircle,
18   Zap,
19 } from "lucide-react";
20
21 const features = [
22   {
23     icon: Upload,
24     title: "Easy Upload",
25     description: "Students can upload copyright filing PDFs with automatic title and filing number extraction",
26     badge: "Core",
27   },
28   {
29     icon: Eye,
30     title: "Smart Access Control",
31     description: "Internal mentors get full access. External reviewers see only abstracts and comments.",
32     badge: "Security",
33   },
34   {
35     icon: Lock,
36     title: "Secure Vault",
37     description: "Government portal credentials are stored securely and only visible to authorized mentors.",
38     badge: "Security",
39   },
40   {
41     icon: FileText,
42     title: "PDF Extraction",
43     description: "Automatic extraction of key information from uploaded copyright receipt documents.",
44     badge: "AI",
45   },
46   {
47     icon: Users,
48     title: "College-Based Routing",
49     description: "Students select their college and mentor during registration for streamlined workflows.",
50     badge: "Workflow",
51   },
52   {
53     icon: Globe,
54     title: "Global Research",
55     description: "Mentors can browse filings across all colleges for cross-institutional research collaborat",
56     badge: "Discovery",
57   },
58 ];

```

Figure 10 – Student Submission Form

```

1 const path = require("path");
2 const express = require("express");
3 const cors = require("cors");
4 const cookieParser = require("cookie-parser");
5 require("dotenv").config();
6
7 const connectDB = require("./config/db");
8 const authRoutes = require("./routes/auth.routes");
9 const copyrightRoutes = require("./routes/copyright.routes");
10 const mentorRoutes = require("./routes/mentor.routes");
11 const notificationRoutes = require("./routes/notification.routes");
12 const profileRoutes = require("./routes/profile.routes");
13 const publicRoutes = require("./routes/public.routes");
14 const plagiarismRoutes = require("./routes/plagiarism.routes");
15 const errorHandler = require("./middlewares/error-handler");
16
17 const app = express();
18
19 const clientOrigin = process.env.CLIENT_ORIGIN || "http://localhost:5173";
20
21 app.use(
22   cors({
23     origin: clientOrigin,
24     credentials: true,
25   })
26 );
27
28 app.use(express.json());
29 app.use(express.urlencoded({ extended: false }));
30 app.use(cookieParser());
31
32 // Serve uploaded files (avatars, etc.)
33 app.use("/uploads", express.static(path.join(__dirname, "uploads")));
34
35 app.get("/api/health", (req, res) => {
36   res.json({ ok: true });
37 });
38
39 app.use("/api/auth", authRoutes);
40 app.use("/api/copyrights", copyrightRoutes);
41 app.use("/api/mentor", mentorRoutes);
42 app.use("/api/notifications", notificationRoutes);
43 app.use("/api/profile", profileRoutes);
44 app.use("/api/public", publicRoutes);
45 app.use("/api/plagiarism", plagiarismRoutes);
46
47 app.use(errorHandler);
48
49 const port = process.env.PORT || 8080;
50
51 async function startServer() {
52   try {
53     await connectDB();
54     app.listen(port, () => {
55       console.log(`Server is running successfully at ${port}`);
56     });
57   } catch (error) {
58     console.error("Failed to start server", error);
59   }
60 }

```

Figure 11 – Student Portfolio with Filing Status

11.5 Backend Code Snapshots

The following code snapshots demonstrate the backend implementation including authentication routes, copyright route handlers, middleware, and database schema definitions.

```

Server > routes > authRoutes.js
1 const express = require("express");
2 const bcrypt = require("bcryptjs");
3
4 const User = require("../models/User");
5 const { requireAuth, requireUser } = require("../middlewares/auth");
6 const { signToken, setAuthToken, clearAuthToken } = require("../utils/auth");
7
8 const router = express.Router();
9
10 function sanitizeUser(user) {
11   return {
12     _id: user._id,
13     name: user.name,
14     email: user.email,
15     role: user.role,
16     college: user.college,
17     mentor: user.mentor || null,
18   };
19 }
20
21 router.post("/register", requireUser, async (req, res, next) => {
22   try {
23     const { name, email, password, role, college, mentor } = req.body;
24
25     if (!name || !email || !password || !role || !college) {
26       return res.status(400).json({ error: "All required fields must be provided" });
27     }
28
29     if (!["student", "mentor"].includes(role)) {
30       return res.status(400).json({ error: "Invalid role" });
31     }
32
33     if (password.length < 6) {
34       return res.status(400).json({ error: "Password must be at least 6 characters" });
35     }
36
37     const existing = await User.findOne({ email: email.toLowerCase() });
38     if (existing) {
39       return res.status(409).json({ error: "Email already registered" });
40     }
41
42     let mentorId = null;
43     if (role === "student" && mentor) {
44       const mentorUser = await User.findOne({ _id: mentor, role: "mentor" });
45       if (!mentorUser) {
46         return res.status(400).json({ error: "Selected mentor does not exist" });
47       }
48       mentorId = mentorUser._id;
49     }
50
51     const passwordHash = await bcrypt.hash(password, 10);
52
53     const user = await User.create({
54       name,
55       email,
56       passwordHash,
57       role,
58       college,
59     });
60
61     signToken(user);
62     setAuthToken(signToken(user));
63     return res.status(201).json({ user: sanitizeUser(user) });
64   } catch (error) {
65     next(error);
66   }
67 });
68
69 router.post("/login", requireUser, async (req, res, next) => {
70   try {
71     const { email, password } = req.body;
72     if (!email || !password) {
73       return res.status(400).json({ error: "Email and password are required" });
74     }
75     const user = await User.findOne({ email: email.toLowerCase() });
76     if (!user) {
77       return res.status(401).json({ error: "Invalid email or password" });
78     }
79     const isValidPassword = await bcrypt.compare(password, user.passwordHash);
80     if (!isValidPassword) {
81       return res.status(401).json({ error: "Invalid email or password" });
82     }
83     signToken(user);
84     setAuthToken(signToken(user));
85     return res.json({ user: sanitizeUser(user) });
86   } catch (error) {
87     next(error);
88   }
89 });
90
91 router.post("/reset-password", requireAuth, async (req, res, next) => {
92   try {
93     const { email, newPassword } = req.body;
94     if (!email || !newPassword) {
95       return res.status(400).json({ error: "Email and new password are required" });
96     }
97     const user = await User.findOne({ email: email.toLowerCase() });
98     if (!user) {
99       return res.status(401).json({ error: "Invalid email or password" });
100     }
101     const passwordHash = await bcrypt.hash(newPassword, 10);
102     await User.updateOne({ _id: user._id }, { passwordHash });
103     return res.status(200).json({ message: "Password reset successful" });
104   } catch (error) {
105     next(error);
106   }
107 });
108
109 module.exports = router;

```

```

Server > routes > copyrightRoutes.js
1 const express = require("express");
2 const multer = require("multer");
3 const path = require("path");
4 const fs = require("fs/promises");
5 const crypto = require("crypto");
6 const { execFile } = require("child_process");
7
8 const Copyright = require("../models/Copyright");
9 const Comment = require("../models/Comment");
10 const Notification = require("../models/Notification");
11 const User = require("../models/User");
12 const { requireAuth, requireRole } = require("../middlewares/auth");
13 const { encryptBuffer, decryptBuffer, encryptBufferForReport, decryptBufferForReport } = require("../utils/file");
14
15 const router = express.Router();
16
17 // Per-field size limits
18 const MAX_RECEIVE_BYTES = 500 * 1024; // 500 KB
19 const MAX_REPORT_BYTES = 2 * 1024 * 1024; // 2 MB
20
21 const upload = multer({
22   storage: multer.memoryStorage(),
23   fileFilter: (req, file, cb) => {
24     if (file.mimetype !== "application/pdf") {
25       return cb(new Error("Only PDF files are allowed"));
26     }
27     return cb(null, true);
28   },
29 });
30 // Set a reasonable global cap (max of per-field limits). We'll also enforce
31 // per-field sizes inside the route handler to provide precise error messages.
32 const limits = {
33   fileSize: MAX_REPORT_BYTES,
34 };
35
36 const PRIVATE_UPLOADS_DIR = path.join(__dirname, "..", "uploads", "private");
37 const RECEIPTS_DIR = path.join(PRIVATE_UPLOADS_DIR, "receipts");
38 const REPORTS_DIR = path.join(PRIVATE_UPLOADS_DIR, "reports");
39
40 function isAuthorizedForAccess(copyrightDoc, user) {
41   const isOwnerStudent = user.role === "student" && String(copyrightDoc.student) === String(user._id);
42   const isAssignedMentor =
43     user.role === "mentor" &&
44     copyrightDoc.mentor &&
45     String(copyrightDoc.mentor) === String(user._id);
46
47   const isGranted = Array.isArray(copyrightDoc.accessGranted)
48     ? copyrightDoc.accessGranted.some((id) => String(id) === String(user._id))
49     : false;
50
51   return isOwnerStudent || isAssignedMentor || isGranted;
52 }
53
54 function resolveLegacyFilePath(filePath) {
55   if (!filePath || typeof filePath !== "string" || !filePath.startsWith("/uploads/")) {
56     return null;
57   }
58 }

```

```

Server > middleware > auth.js
1 const jwt = require("jsonwebtoken");
2 const User = require("../models/User");
3 const { signToken, setAuthToken } = require("../utils/auth");
4
5 async function getHeaderFromRequest(req) {
6   const tokenFromCookie = req.cookies.token;
7   const tokenFromHeader = req.headers.authorization?.startsWith("Bearer ")
8     ? req.headers.authorization.split(" ")[1]
9     : null;
10
11   const token = tokenFromCookie || tokenFromHeader;
12
13   if (!token) {
14     return { user: null, tokenFromCookie: null };
15   }
16
17   const payload = jwt.verify(token, process.env.JWT_SECRET);
18   const user = await User.findById(payload.id).select("-passwordHash");
19   return { user, tokenFromCookie };
20 }
21
22 const { user, tokenFromCookie } = await getHeaderFromRequest(req);
23
24 if (!user) {
25   return res.status(401).json({ error: "Unauthorized" });
26 }
27
28 req.user = user;
29
30 // Rolling session: refresh cookie for active users, expire for inactive users
31 if (tokenFromCookie) {
32   const refreshToken = signToken(user);
33   setAuthToken(refreshToken);
34 }
35
36 return next();
37
38 } catch (error) {
39   return res.status(401).json({ error: "Unauthorized" });
40 }
41
42
43
44
45 function requireRole(...roles) {
46   return (req, res, next) => {
47     if (!req.user || !roles.includes(req.user.role)) {
48       return res.status(403).json({ error: "Forbidden" });
49     }
50
51     return next();
52   };
53 }
54
55 async function requireAuth(req, res, next) {
56   try {
57     const { user } = await getHeaderFromRequest(req);
58     if (!user) {
59       return res.status(401).json({ error: "Unauthorized" });
60     }
61     req.user = user;
62     next();
63   } catch (error) {
64     next(error);
65   }
66 }

```

```

Server > models > user.js
1 const mongoose = require("mongoose");
2
3 const userSchema = new mongoose.Schema(
4   {
5     name: {
6       type: String,
7       required: true,
8       trim: true,
9     },
10    email: {
11      type: String,
12      required: true,
13      unique: true,
14      lowercase: true,
15      trim: true,
16    },
17    passwordHash: {
18      type: String,
19      required: true,
20    },
21    roles: {
22      type: String,
23      enum: ["student", "mentor"],
24      required: true,
25    },
26    college: {
27      type: String,
28      required: true,
29      trim: true,
30    },
31    avatarUrl: {
32      type: String,
33      default: "",
34    },
35    mentor: {
36      type: mongoose.Schema.Types.ObjectId,
37      ref: "User",
38      default: null,
39    },
40    timestamps: {
41      type: Boolean,
42      default: true,
43    },
44  },
45   { timestamps: true }
46 );
47
48 module.exports = mongoose.model("User", userSchema);

```

```

Server > models > comment.js
1 const mongoose = require("mongoose");
2
3 const commentSchema = new mongoose.Schema(
4   {
5     copyright: {
6       type: mongoose.Schema.Types.ObjectId,
7       ref: "Copyright",
8       required: true,
9     },
10    mentor: {
11      type: mongoose.Schema.Types.ObjectId,
12      ref: "User",
13      required: true,
14    },
15    text: {
16      type: String,
17      required: true,
18      trim: true,
19    },
20    timestamps: {
21      type: Boolean,
22      default: true,
23    },
24  },
25   { timestamps: true }
26 );
27
28 module.exports = mongoose.model("Comment", commentSchema);

```

Figure 12 – Auth Routes & Copyright Routes (Backend)

```

Server > middleware > auth.js
1 const jwt = require("jsonwebtoken");
2 const User = require("../models/User");
3 const { signToken, setAuthToken } = require("../utils/auth");
4
5 async function getHeaderFromRequest(req) {
6   const tokenFromCookie = req.cookies.token;
7   const tokenFromHeader = req.headers.authorization?.startsWith("Bearer ")
8     ? req.headers.authorization.split(" ")[1]
9     : null;
10
11   const token = tokenFromCookie || tokenFromHeader;
12
13   if (!token) {
14     return { user: null, tokenFromCookie: null };
15   }
16
17   const payload = jwt.verify(token, process.env.JWT_SECRET);
18   const user = await User.findById(payload.id).select("-passwordHash");
19   return { user, tokenFromCookie };
20 }
21
22 const { user, tokenFromCookie } = await getHeaderFromRequest(req);
23
24 if (!user) {
25   return res.status(401).json({ error: "Unauthorized" });
26 }
27
28 req.user = user;
29
30 // Rolling session: refresh cookie for active users, expire for inactive users
31 if (tokenFromCookie) {
32   const refreshToken = signToken(user);
33   setAuthToken(refreshToken);
34 }
35
36 return next();
37
38 } catch (error) {
39   return res.status(401).json({ error: "Unauthorized" });
40 }
41
42
43
44
45 function requireRole(...roles) {
46   return (req, res, next) => {
47     if (!req.user || !roles.includes(req.user.role)) {
48       return res.status(403).json({ error: "Forbidden" });
49     }
50
51     return next();
52   };
53 }
54
55 async function requireAuth(req, res, next) {
56   try {
57     const { user } = await getHeaderFromRequest(req);
58     if (!user) {
59       return res.status(401).json({ error: "Unauthorized" });
60     }
61     req.user = user;
62     next();
63   } catch (error) {
64     next(error);
65   }
66 }

```

```

Server > models > user.js
1 const mongoose = require("mongoose");
2
3 const userSchema = new mongoose.Schema(
4   {
5     name: {
6       type: String,
7       required: true,
8       trim: true,
9     },
10    email: {
11      type: String,
12      required: true,
13      unique: true,
14      lowercase: true,
15      trim: true,
16    },
17    passwordHash: {
18      type: String,
19      required: true,
20    },
21    roles: {
22      type: String,
23      enum: ["student", "mentor"],
24      required: true,
25    },
26    college: {
27      type: String,
28      required: true,
29      trim: true,
30    },
31    avatarUrl: {
32      type: String,
33      default: "",
34    },
35    mentor: {
36      type: mongoose.Schema.Types.ObjectId,
37      ref: "User",
38      default: null,
39    },
40    timestamps: {
41      type: Boolean,
42      default: true,
43    },
44  },
45   { timestamps: true }
46 );
47
48 module.exports = mongoose.model("User", userSchema);

```

```

Server > models > comment.js
1 const mongoose = require("mongoose");
2
3 const commentSchema = new mongoose.Schema(
4   {
5     copyright: {
6       type: mongoose.Schema.Types.ObjectId,
7       ref: "Copyright",
8       required: true,
9     },
10    mentor: {
11      type: mongoose.Schema.Types.ObjectId,
12      ref: "User",
13      required: true,
14    },
15    text: {
16      type: String,
17      required: true,
18      trim: true,
19    },
20    timestamps: {
21      type: Boolean,
22      default: true,
23    },
24  },
25   { timestamps: true }
26 );
27
28 module.exports = mongoose.model("Comment", commentSchema);

```

Figure 13 – Auth Middleware, User Schema & Comment Schema

11.6 Frontend Code Snapshots

The following frontend code snapshots show the React TypeScript component implementations including the comment section, copyright card rendering, access request management, and application routing structure.

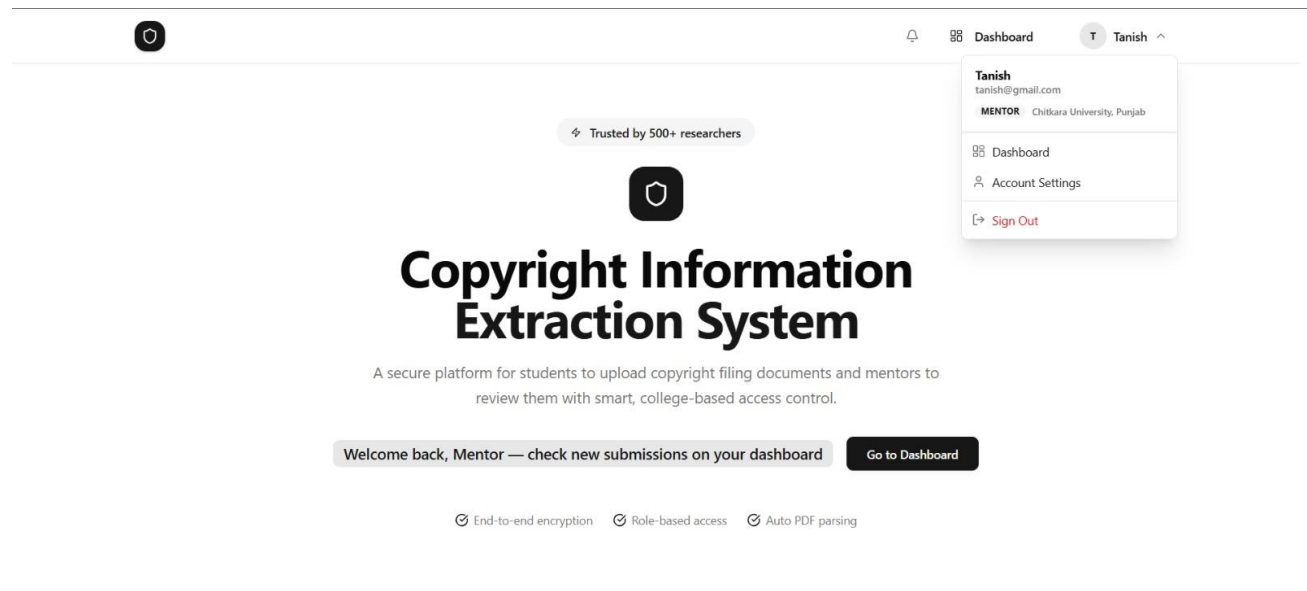


Figure 14 – Comment Section & Copyright Card Components (Frontend)

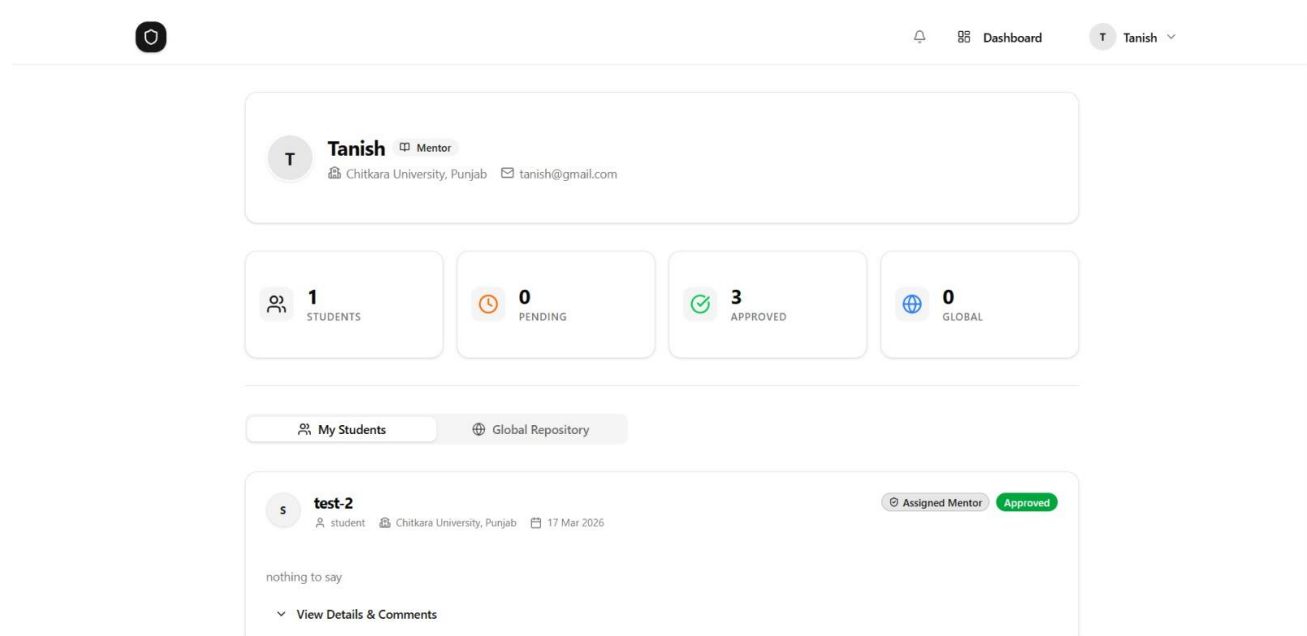


Figure 15 – Access Requests & Copyright List Components

The screenshot displays the landing page of the CIE-Vault application. At the top, there is a navigation bar with a logo on the left, a bell icon, a 'Dashboard' link, and a user profile dropdown labeled 'student'. Below the navigation bar, the user profile is shown with a circular avatar containing the letter 'S', the name 'student', and the email 'student@gmail.com'. The main content area features a 'New Submission' section with a 'Submit Copyright Filing' button and a 'PDF Only' toggle. The form includes fields for 'Project Title' (with a placeholder 'e.g., Blockchain-based supply chain system'), 'Abstract / Summary' (with a placeholder 'Describe your research or project work briefly...'), 'Your Institution', and 'Assigned Mentor'.

Figure 16 – Landing Page Component & App Routing (App.tsx)

The screenshot shows the submission and portfolio section of the application. It features a 'Click to upload or drag and drop' area with a note 'Receipt will be processed for AI verification'. Below this is a 'Submit for Mentor Review' button. The 'Your Portfolio' section displays a table titled 'Your Copyright Filings' with 4 entries.

Document Title	Filing Number	Assigned Mentor	Date	Status
test-2	-	Tanish	17 Mar 2026	Approved
check for the test	-	Tanish	17 Mar 2026	Approved
Check the security of the modeule	-	Tanish	17 Mar 2026	Approved
sdfgsdads	-	Tanish	16 Mar 2026	Rejected

Figure 17 – Server.js Entry Point & Database Configuration

12. COPYRIGHT RECEIPT DETAILS

The CIE-Vault project was officially filed through the Government of India Copyright Office. The following are the official filing details:

Field	Details
Receipt Number	239643
Filing Date	24/04/2026
Diary Number	SW-19434/2026-CO
Request Number	271486
Title	CIE-Vault

P.T.O

13. EXPECTED OUTCOMES AND BENEFITS

13.1 Centralized Research Management

The system provides a unified platform for handling all institutional research and copyright records, eliminating fragmented document storage and enabling easy retrieval.

13.2 Enhanced Security

AES-256 encryption, JWT authentication, and role-based access control significantly improve intellectual property protection and reduce unauthorized access risks.

13.3 Reduced Manual Work

Automated metadata extraction using pdf-parse and OCR reduces repetitive administrative tasks and minimizes data entry errors across the institution.

13.4 Improved Academic Integrity

The hybrid plagiarism detection engine combining Jaccard similarity and Gemini AI analysis discourages copied or AI-generated content in academic submissions.

13.5 Better Collaboration

The notification and workflow systems significantly improve communication between students and mentors, ensuring timely feedback and transparent approval tracking.

13.6 Scalable Infrastructure

The modular architecture supports future expansion, allowing integration of new features, cloud storage, mobile applications, and advanced AI capabilities without disrupting existing workflows.

13.7 Transparent Approval Workflow

Students can track approval status, mentor feedback, and submission history in real-time through their personal dashboard, creating institutional accountability.

14. CONCLUSION AND FUTURE SCOPE

14.1 Conclusion

CIE-Vault was successfully developed as a secure, scalable, and intelligent academic copyright management platform aimed at solving the limitations of traditional research filing workflows.

The system integrates automation, encryption, workflow management, plagiarism analysis, and institutional collaboration into a single cohesive ecosystem. Through modern MERN-stack technologies, the platform enables secure upload, storage, validation, review, and controlled sharing of research documents.

The implementation of AES-256 encryption, JWT authentication, hybrid access control, and secure document previews ensures that intellectual property remains protected throughout the complete workflow lifecycle. Automated metadata extraction significantly reduces manual effort while the plagiarism detection engine strengthens academic integrity.

Overall, CIE-Vault fulfills its objective of creating a centralized and secure academic copyright filing ecosystem suitable for educational institutions of all scales.

14.2 Future Scope

- **AI-Based Research Evaluation:** Future systems may support research quality scoring, AI mentor suggestions, and semantic categorization of submitted work.
- **Blockchain-Based Verification:** Blockchain integration may provide immutable ownership records, transparent audit trails, and decentralized verification mechanisms.
- **Mobile Application:** A mobile version can support real-time notifications, secure uploads, and portable workflow management for on-the-go access.
- **Multi-University Integration:** The system may evolve into an inter-university collaborative research ecosystem with shared repositories.
- **Cloud Storage Integration:** Future versions may integrate AWS S3, Google Cloud Storage, or Azure Blob Storage for scalable and redundant file storage.
- **Advanced AI Plagiarism Detection:** Future AI models may support cross-language similarity checks, deep semantic analysis, and AI-generated content detection.
- **Real-Time Collaboration:** Shared annotations, research forums, and real-time editing capabilities may be added for collaborative document review.

15. REFERENCES

1. National Institute of Standards and Technology (NIST). (2001). FIPS Publication 197: Advanced Encryption Standard (AES). U.S. Department of Commerce.
2. Jones, M., Bradley, J., & Sakimura, N. (2015). RFC 7519: JSON Web Token (JWT). Internet Engineering Task Force (IETF). <https://datatracker.ietf.org/doc/html/rfc7519>
3. Chodorow, K. (2013). MongoDB: The Definitive Guide. O'Reilly Media.
4. Rescorla, E. (2018). The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446.
5. OWASP Foundation. (2025). Node.js Security Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Nodejs_Security_Cheat_Sheet.html